

Atividade Prática — Docker + Compose + CI

Entrega: até o início da próxima aula **Formato:** link de um repositório público no GitHub + README preenchido com prints (imagens) comprovando a execução **Trabalho:** individual ou em dupla (a definir pelo professor)

1. Objetivo

Você **não vai programar a aplicação**. A aplicação já existe e está pronta. Sua missão é **empacotar (encapsular) essa aplicação em containers** e provar que você entendeu:

- Imagens e Dockerfile (com **multi-stage build**)
 - Volumes (persistência de dados)
 - Redes (containers conversando entre si)
 - Docker Compose (orquestração de múltiplos serviços)
 - CI básico com GitHub Actions (incluindo **ver o pipeline falhar e depois consertar**)
-

2. A aplicação base

Vamos usar o app oficial de exemplo da Docker: um **gerenciador de tarefas (To-Do)** em Node.js — um CRUD completo (criar, listar, editar, excluir tarefas) com interface web.

```
git clone https://github.com/docker/getting-started-app.git meu-projeto-docker
cd meu-projeto-docker
rm -rf .git # Windows/PowerShell: Remove-Item -Recurse -Force .git
git init
```

Importante: remova a pasta `.git` original e inicialize um repositório novo, senão você vai estar carregando o histórico do repositório da Docker.

O que você precisa saber sobre o app (e só isso):

Item	Valor

Runtime	Node.js 18+
Comando de inicialização	<code>node src/index.js</code>
Porta interna	3000
Banco padrão	SQLite, gravado em <code>/etc/todos/todo.db</code>
Banco alternativo	MySQL, ativado via variáveis de ambiente
Endpoint de API	<code>GET /items</code> (lista as tarefas em JSON) e <code>POST /items</code>

Variáveis de ambiente para usar MySQL:

```

MYSQL_HOST=<nome do serviço do banco>
MYSQL_USER=<usuário>
MYSQL_PASSWORD=<senha>
MYSQL_DB=<nome do banco>

```

Se `MYSQL_HOST` não estiver definida, o app usa SQLite automaticamente. Isso é útil: a Parte 2 usa SQLite (mais simples) e a Parte 3 migra para MySQL.

Alternativa (opcional): se preferir outra stack (Python/Flask, Go, PHP), você pode escolher um app pronto do repositório [docker/awesome-compose](#). Nesse caso **avise o professor antes** e adapte todas as etapas abaixo. Os requisitos de entrega são os mesmos.

3. Roteiro da atividade

Parte 1 — Dockerfile multi-stage

Crie um `Dockerfile` na raiz do projeto com **dois estágios**:

- Estágio de build** (ex.: `FROM node:20-alpine AS builder`): copia os arquivos de dependência (`package.json` e o lockfile), instala as dependências.
- Estágio final** (ex.: `FROM node:20-alpine`): copia **apenas** o necessário do estágio anterior (`node_modules` + código-fonte), define `WORKDIR`, `EXPOSE 3000`, `USER não-root` e `CMD`.

Requisitos obrigatórios:

- Usar `AS <nome>` e `COPY --from=<nome>` (é isso que caracteriza o multi-stage)

- Instalar dependências **antes** de copiar o código todo (aproveitamento de cache de camadas)
- Criar um arquivo `.dockerignore` contendo, no mínimo: `node_modules`, `.git`, `*.md`
- A imagem final **não** deve rodar como `root`

Construa e rode:

```
docker build -t todo-app:v1 .
docker run -d -p 3000:3000 --name todo todo-app:v1
```

Acesse `http://localhost:3000`, cadastre 2 ou 3 tarefas e tire um print.

Evidência extra pedida: rode `docker images` e mostre o tamanho da sua imagem.

Comente no README uma frase sobre por que o multi-stage ajuda no tamanho/segurança da imagem final.

Parte 2 — Volume e persistência (containers avulsos)

Prove que **sem volume os dados somem e com volume eles sobrevivem**.

```
# 1) Sem volume: crie tarefas, depois destrua o container
docker rm -f todo
docker run -d -p 3000:3000 --name todo todo-app:v1
# ...cadastre tarefas no navegador...
docker rm -f todo
docker run -d -p 3000:3000 --name todo todo-app:v1
# ...abra o navegador: a lista está vazia. PRINT AQUI.

# 2) Com volume nomeado
docker rm -f todo
docker volume create todo-db
docker run -d -p 3000:3000 -v todo-db:/etc/todos --name todo todo-app:v1
# ...cadastre tarefas...
docker rm -f todo
docker run -d -p 3000:3000 -v todo-db:/etc/todos --name todo todo-app:v1
# ...as tarefas continuam lá. PRINT AQUI.
```

Evidência: dois prints do navegador (antes/depois) + saída de `docker volume ls`.

Parte 3 — Rede (containers conversando)

Agora o app vai usar MySQL em vez de SQLite, **sem Compose ainda**, para você ver a rede funcionando “na mão”.

```
docker network create todo-net
```

```
docker run -d \
  --network todo-net --network-alias mysql \
  -v todo-mysql-data:/var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=secret \
  -e MYSQL_DATABASE=todos \
  --name mysql mysql:8.0
```

```
docker run -d -p 3000:3000 \
  --network todo-net \
  -e MYSQL_HOST=mysql \
  -e MYSQL_USER=root \
  -e MYSQL_PASSWORD=secret \
  -e MYSQL_DB=todos \
  --name todo todo-app:v1
```

Valide:

```
docker exec -it mysql mysql -p todos      # senha: secret
# dentro do MySQL:
select * from todo_items;
```

Evidências: saída de `docker network inspect todo-net` (mostrando os dois containers conectados) e o `select` retornando as tarefas que você cadastrou pelo navegador.

Responda no README, em uma frase: **por que o app consegue chamar o host `mysql` sem saber o IP dele?**

Ao final, limpe tudo: `docker rm -f todo mysql`

Parte 4 — Docker Compose

Crie um `compose.yaml` que suba tudo com **um comando só**. Requisitos:

- Dois serviços: `app` (build a partir do seu Dockerfile) e `db` (imagem `mysql:8.0`)
- Uma **rede** nomeada explicitamente
- Um **volume nomeado** para os dados do MySQL

- Variáveis de ambiente do app apontando para o serviço `db`
- A porta do **banco NÃO deve ser publicada** para o host (só o app precisa acessá-lo)
- `healthcheck` no `db` + `depends_on` com `condition: service_healthy` no `app` (evita o app subir antes do banco estar pronto)
- Senhas em um arquivo `.env` (e o `.env` listado no `.gitignore` ; entregue um `.env.example` no lugar)

```
docker compose config      # valida o arquivo
docker compose up -d --build
docker compose ps
docker compose logs app
```

Teste de persistência obrigatório:

```
# cadastre tarefas no navegador
docker compose down          # SEM -v: derruba containers, mantém o volume
docker compose up -d
# as tarefas devem continuar lá -> PRINT
docker compose down -v      # AGORA sim apaga o volume
docker compose up -d
# a lista deve estar vazia    -> PRINT
```

Explique no README, em uma frase, a diferença entre `docker compose down` e `docker compose down -v`.

Parte 5 — CI com GitHub Actions

Crie o arquivo `.github/workflows/ci.yml` . O pipeline deve rodar a cada `push` e `pull_request` e fazer:

1. Validar o `compose.yaml` (`docker compose config`)
2. Buildar a imagem
3. Subir a stack
4. Fazer um **smoke test**: esperar a aplicação responder e criar uma tarefa via API
5. Derrubar a stack

Use este esqueleto — os pontos marcados com `TODO` são seus:

name: CI

on:

push:

pull_request:

jobs:

build-and-test:

runs-on: ubuntu-latest

steps:

- name: Checkout do código
uses: actions/checkout@v4

- name: Criar .env para o CI
run: cp .env.example .env

- name: Validar o compose
run: docker compose config

- name: Build da imagem
run: docker compose build # TODO: confira se o nome do serviço bate

- name: Subir a stack
run: docker compose up -d

- name: Aguardar a aplicação responder
run: |
for i in \$(seq 1 30); do
if curl -sf http://localhost:3000/items > /dev/null; then
echo "Aplicacao no ar!"
exit 0
fi
echo "tentativa \$i..."
sleep 3
done
echo "A aplicacao nao subiu a tempo"
docker compose logs
exit 1

- name: Smoke test do CRUD
run: |
curl -sf -X POST http://localhost:3000/items \
-H 'Content-Type: application/json' \
-d '{"id":"ci-test-1","name":"tarefa criada pelo CI","completed":fals
echo ""
curl -sf http://localhost:3000/items | tee resposta.json

```
grep -q "tarefa criada pelo CI" resposta.json
```

```
- name: Logs em caso de falha
  if: failure()
  run: docker compose logs

- name: Derrubar a stack
  if: always()
  run: docker compose down -v
```

Evidência: print da aba **Actions** com a execução  verde e print dos logs do job.

Parte 6 — Quebrar o CI de propósito (obrigatório)

Você precisa mostrar que entende que o CI **serve para pegar erro**. Faça o pipeline falhar de propósito, tire o print vermelho, e depois conserte.

Escolha **uma** das quebras abaixo (ou invente a sua):

#	O que quebrar	Resultado esperado
1	Errar o caminho no <code>CMD</code> do Dockerfile (ex.: <code>src/indexx.js</code>)	container sobe e morre, smoke test estoura
2	Remover a variável <code>MYSQL_PASSWORD</code> do <code>compose</code>	app não conecta no banco
3	Trocar a URL do smoke test para uma rota inexistente (<code>/itemsss</code>)	<code>curl -sf</code> retorna erro
4	Errar a indentação/uma chave inválida no <code>compose.yaml</code>	<code>docker compose config</code> falha logo no começo

Fluxo esperado (faça exatamente assim, é isso que comprova o aprendizado):

1. Crie uma branch: `git checkout -b quebra-proposital`
2. Aplique a quebra, commit e push
3. Abra um **Pull Request** para a `main` → o Actions roda e fica  vermelho
4. **Print** da execução vermelha + **print do trecho do log que mostra o erro**
5. Corrija na mesma branch, commit e push

6. **Print** da execução  verde

7. Faça o merge do PR

No README, escreva 2 ou 3 frases: **o que você quebrou, como o CI reagiu e como você descobriu o problema pelos logs.**

4. O que deve estar no repositório

```
meu-projeto-docker/
├── .github/
│   └── workflows/
│       └── ci.yml
├── docs/
│   └── imagens/                <- todos os prints ficam aqui
├── src/                       <- código da aplicação (não mexa)
├── .dockerignore
├── .env.example
├── .gitignore
├── compose.yaml
├── Dockerfile
├── package.json
└── README.md                  <- preenchido pelo template
```

Não commitar: `node_modules/` , `.env` , arquivos de banco locais.

Como colocar as imagens no README: salve os prints em `docs/imagens/` e referencie assim:

```
![Aplicação rodando] (docs/imagens/01-app-rodando.png)
```

5. Checklist de entrega

- Repositório **público** no GitHub com histórico de commits (não um único commit "final")
- `Dockerfile` multi-stage funcional + `.dockerignore`
- `compose.yaml` com rede, volume nomeado, variáveis de ambiente e healthcheck
- `.env.example` versionado e `.env` ignorado
- Workflow do GitHub Actions funcionando

- Um PR com o CI vermelho e depois verde (histórico visível)
- README preenchido com **todos** os prints pedidos e as respostas das perguntas
- Link do repositório enviado ao professor

6. Prints obrigatórios (mínimo 9)

#	Print
1	<code>docker build</code> concluído + <code>docker images</code> mostrando o tamanho da imagem
2	App rodando no navegador com tarefas cadastradas
3	Perda de dados sem volume (lista vazia após recriar o container)
4	Persistência com volume (lista intacta após recriar o container)
5	<code>docker network inspect</code> com os dois containers na mesma rede
6	<code>select * from todo_items;</code> dentro do container MySQL
7	<code>docker compose ps</code> com todos os serviços de pé
8	Execução do GitHub Actions  verde
9	Execução do GitHub Actions  vermelha + trecho do log com o erro

7. Critérios de avaliação (10 pontos)

Critério	Pontos
Dockerfile multi-stage correto, com <code>.dockerignore</code> e usuário não-root	2,0
Volume e demonstração clara de persistência	1,5
Rede: containers se comunicando por nome, banco não exposto ao host	1,5
Compose completo e funcional (<code>up -d --build</code> sobe tudo de primeira)	2,0
CI verde com smoke test real da aplicação	1,5

Quebra proposital documentada (PR vermelho → verde + explicação)	1,0
README organizado, prints legíveis e perguntas respondidas	0,5

Descontos: segredo commitado (`.env` , senha em código) $-1,0 \cdot \text{node_modules}$ commitado $-0,5 \cdot \text{README sem prints}$ $-2,0$

8. Erros comuns (leia antes de pedir ajuda)

Sintoma	Provável causa
<code>ECONNREFUSED</code> do app para o banco	banco ainda subindo — falta <code>healthcheck</code> + <code>depends_on: condition: service_healthy</code>
Porta 3000 ocupada	outro container antigo rodando → <code>docker ps</code> e <code>docker rm -f <nome></code>
Dados somem no <code>compose up</code>	you used <code>down -v</code> sem querer, ou não declarou o volume no serviço
Build muito lento toda vez	you copied the whole code before <code>npm install</code> — inverta a ordem
CI falha no <code>.env</code>	lack the step <code>cp .env.example .env</code> no workflow
App não acha <code>mysql</code>	service with different name in compose or out of the same network
Imagem gigante (1GB+)	lack <code>.dockerignore</code> or you copied the whole build stage

Comandos de socorro:

```
docker compose logs -f app      # ver o que o app está reclamando
docker compose ps              # status dos serviços
docker exec -it <container> sh  # entrar no container
docker system prune -a         # limpeza geral (cuidado: apaga imagens)
```